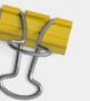


생성형 AI와 LLM 기반 AI 에이전트 개발

문제 정의 및 에이전트 설계서 작성



왜 설계가 먼저인가?

문제 정의 및 에이전트 설계서 작성

- 실제 구현을 염두에 두고 더 구체적으로 설계하는 과정 연습
 - 좋은 에이전트는 코딩보다 설계가 먼저임. 설계 없이 바로 코딩하면:
 - 만들다 보면 방향이 바뀌어 처음부터 다시 짜게 됨
 - 어떤 Tool이 필요한지 나중에야 알게 되어 구조가 엉킴
 - 결과물이 "뭘 하는 에이전트인지" 불명확해짐
 - 반면 설계를 먼저 하면:
 - 구현 전에 전체 구조가 머릿속에 잡혀 있어 코딩 속도가 빠름
 - 필요한 Tool, API, 라이브러리를 미리 파악해서 준비 가능
 - 결과물의 목적과 범위가 명확해져 완성도가 높아짐
- ➔ AI 에이전트 개발에서 설계 능력은 코딩 능력만큼 중요한 핵심 역량.

문제 정의: 무엇을 만들 것인가?

문제 정의 및 에이전트 설계서 작성

- 좋은 에이전트를 만들려면 먼저 해결하려는 문제를 명확하게 정의해야 함.
문제 정의가 모호하면 에이전트의 방향도 흐려짐.
- 문제 정의 시 답해야 할 질문:
 - 누가 쓰는가? → 타겟 사용자 정의
 - 어떤 문제를 해결하는가? → 해결하려는 핵심 문제
 - 지금은 어떻게 해결하고 있는가? → 기존 방식의 한계
 - 에이전트가 있으면 무엇이 달라지는가? → 기대 효과
- 좋은 문제 정의 예시:

❌ "취업을 도와주는 에이전트를 만들겠다"
→ 범위가 너무 넓고 모호함

○ "취업 준비 중인 대학생이 채용공고를 붙여넣으면
자소서 부족부분을 찾아주고 개선안을 제시해주는
에이전트를 만들겠다"
→ 사용자, 문제, 기대 효과가 명확함

에이전트 설계서 작성

문제 정의 및 에이전트 설계서 작성

- 문제 정의가 완료되면 4가지 구성요소 기준으로 설계서 작성.

- 에이전트 이름: _____

에이전트 목표: _____

타겟 사용자: _____

구성요소	내용
LLM	어떤 모델? system_instruction은 어떻게 설정할 것인가?
Tool	어떤 기능(함수)이 필요한가? 외부 API가 필요한가?
Memory	무엇을 기억해야 하는가? 대화 기록 외에 저장할 데이터가 있는가?
Workflow	어떤 순서로 작동하는가? 조건 분기가 필요한가?

- 구현 계획:

- 사용할 라이브러리: _____

- 필요한 외부 API: _____

- LangChain / LangGraph 중 어느 것을 사용할 것인가? _____

LangChain vs LangGraph 선택 기준

문제 정의 및 에이전트 설계서 작성

- 최종 프로젝트에서 LangChain과 LangGraph 중 어느 것을 쓸지 판단하는 기준.

상황	선택
단순한 LLM 호출 + 프롬프트 관리	LangChain Chain
Memory + Tool이 필요한 에이전트	LangChain AgentExecutor
조건 분기나 반복이 필요한 복잡한 흐름	LangGraph
여러 에이전트가 협업하는 구조	LangGraph (멀티 에이전트)

- 앞으로 만들 두 가지 프로젝트:
 - 면접 준비 에이전트: 자소서 분석 + 예상 질문 생성 + 답변 피드백 → LangChain AgentExecutor
 - AI 학습 플래너 서비스: 여러 기능이 통합된 복잡한 구조 → LangGraph

설계서 작성 실습

문제 정의 및 에이전트 설계서 작성

- 지금까지 배운 내용을 바탕으로 나만의 에이전트 설계서를 작성해보세요.
- 주제 선택 가이드:
 - 프로젝트 1(면접 준비 에이전트) 설계
 - 프로젝트 2(AI 학습 플래너 서비스) 설계
 - 또는 완전히 새로운 아이디어로 자유롭게 설계
- 설계하고 검토해볼 부분
 - "이 Tool이 왜 필요한가?"
 - "Workflow 순서가 자연스러운가?"
 - "LangChain과 LangGraph 선택이 적절한가?"

프로젝트 1 : 면접 준비 에이전트 구현



Streamlit이란?

프로젝트 1 : 면접 준비 에이전트 구현

- Streamlit은 Python 코드만으로 웹 애플리케이션을 만들 수 있는 프레임워크. HTML, CSS, JavaScript 없이도 AI 서비스를 웹 UI로 빠르게 구현할 수 있어 AI 에이전트 프로토타입 제작에 널리 활용됨.
- Streamlit의 특징:
 - Python 코드만으로 웹 UI 구현 가능
 - 텍스트 입력, 버튼, 파일 업로드 등 다양한 UI 컴포넌트 제공
 - 코드 변경 시 브라우저에서 즉시 반영
 - Streamlit Cloud를 통해 무료 배포 가능 (30차시)

```
# Streamlit 및 ngrok 설치  
pip install streamlit pyngrok
```

```
# 설치확인  
streamlit --version
```

Streamlit이란?

프로젝트 1 : 면접 준비 에이전트 구현

- **Streamlit은 Python 코드만으로 웹 애플리케이션을 만들 수 있는 프레임워크. HTML, CSS, JavaScript 없이도 AI 서비스를 웹 UI로 빠르게 구현할 수 있어 AI 에이전트 프로토타입 제작에 널리 활용됨.**

```
# Streamlit 기본 예시
```

```
import streamlit as st
```

```
st.title("면접 준비 에이전트")
```

```
user_input = st.text_input("질문을 입력하세요")
```

```
if st.button("전송"):
```

```
    st.write(f"입력한 내용: {user_input}")
```

```
# Streamlit 실행
```

```
streamlit run streamlitEx.py
```

```
# → 브라우저에서 http://localhost:8501 자동으로 열림
```

Streamlit 주요 컴포넌트

프로젝트 1 : 면접 준비 에이전트 구현

- 실습에서 사용할 주요 Streamlit 컴포넌트.

```
import streamlit as st

# 제목과 설명
st.title("면접 준비 에이전트")
st.markdown("자소서와 채용공고를 입력하고 면접 준비를 시작하세요!")

# 긴 텍스트 입력
resume = st.text_area("자소서를 입력하세요", height=200)
job_posting = st.text_area("채용공고를 입력하세요", height=200)

# 버튼
if st.button("정보 저장"):
    st.success("저장되었습니다!")

# 사이드바
with st.sidebar:
    st.header("정보 입력")
    ...
```

주입용 자료 | 무단 배포 금지

Streamlit 주요 컴포넌트

프로젝트 1 : 면접 준비 에이전트 구현

- 실습에서 사용할 주요 Streamlit 컴포넌트.

```
...  
# 채팅 메시지  
with st.chat_message("user"):  
    st.write("면접 질문 생성해줘")  
  
with st.chat_message("assistant"):  
    st.write("예상 면접 질문을 생성했습니다...")  
  
# 채팅 입력  
user_input = st.chat_input("질문을 입력하세요")  
  
# 스피너  
with st.spinner("AI가 분석 중입니다..."):  
    pass  
  
# session_state: 대화 기록 유지  
if "messages" not in st.session_state:  
    st.session_state.messages = []
```

주입용 자료 | 무단 배포 금지

오늘 만들 것

프로젝트 1 : 면접 준비 에이전트 구현

- 자소서와 채용공고를 입력하면 예상 면접 질문을 생성하고, 답변을 작성하면 피드백을 제공하는 면접 준비 에이전트를 Streamlit 웹 서비스로 구현.
- 이전에 만들었던 자소서 리뷰 에이전트에서 한 단계 발전:
 - 자소서 리뷰 → 예상 면접 질문 생성까지 확장
 - 학생이 답변을 작성하면 AI가 피드백 제공
 - Streamlit으로 웹 UI 구현 → 브라우저에서 바로 사용 가능
 - ngrok으로 URL 공유 → 다른 사람도 접속 가능

- **완성 예시:**

[사이드바]

자소서: [입력]

채용공고: [입력]

[채팅 화면]

사용자: 면접 질문 생성해줘

에이전트: 예상 면접 질문 5가지를 생성했습니다.

1. 지원 동기가 무엇인가요?

2. 팀 프로젝트 경험을 말씀해주세요...

사용자: 1번 질문 답변 피드백해줘

에이전트: STAR 기법 기준으로 피드백드립니다...

에이전트 설계

프로젝트 1 : 면접 준비 에이전트 구현

- **에이전트 이름: 면접 준비 에이전트**

에이전트 목표: 자소서와 채용공고를 분석해 예상 면접 질문을 생성하고 답변 피드백 제공

타겟 사용자: 취업 준비 중인 대학생

구성요소	내용
LLM	gpt-4o-mini / "당신은 10년 경력의 대기업 인사담당자이자 면접 코치입니다. 자소서와 채용공고를 분석해 실제 면접에서 나올 법한 질문을 생성하고 답변에 구체적인 피드백을 제공합니다."
Tool	analyze_resume_and_job(자소서·채용광고 분석), generate_interview_questions(면접 질문 생성), feedback_answer(답변 피드백)
Memory	자소서, 채용광고, 생성된 질문, 답변 기록 유지 (ConversationBufferMemory)
Workflow	자소서·채용광고 입력 → 분석 → 질문 생성 → 답변 입력 → 피드백 → 반복

- **구현 계획:**

- 사용할 라이브러리: langchain, langchain-openai, python-dotenv
- 필요한 외부 API: OpenAI API
- 구현 방식: LangChain AgentExecutor + Streamlit (웹 UI)

Tool 설계

프로젝트 1 : 면접 준비 에이전트 구현

■ 면접 준비 에이전트의 핵심 Tool 세 가지.

```
from langchain.tools import tool
```

```
@tool
```

```
def analyze_resume_and_job(resume: str, job_posting: str) -> str:
    """자소서와 채용공고를 분석해 핵심 키워드와 역량을 추출하는 Tool"""
    return f"분석 결과: 핵심 역량 - Python, 데이터 분석, 협업"
```

```
@tool
```

```
def generate_interview_questions(analysis: str) -> str:
    """분석 결과를 바탕으로 예상 면접 질문 5가지를 생성하는 Tool"""
    return """
    1. 지원 동기가 무엇인가요?
    2. 가장 어려웠던 프로젝트 경험을 말씀해주세요.
    3. Python을 활용한 데이터 분석 경험이 있나요?
    4. 팀 내 갈등을 해결한 경험이 있나요?
    5. 입사 후 3년 내 목표는 무엇인가요?
    """
```

```
@tool
```

```
def feedback_answer(question: str, answer: str) -> str:
    """면접 질문에 대한 답변을 분석하고 피드백을 제공하는 Tool"""
    return "피드백: STAR 기법으로 구조화하면 더 효과적입니다."
```

```
tools = [analyze_resume_and_job, generate_interview_questions, feedback_answer]
```

주입용 자료 | 무단 배포 금지

system_instruction 설계

프로젝트 1 : 면접 준비 에이전트 구현

- **면접관 역할을 구체적으로 정의할수록 더 실용적인 피드백이 생성됨.**

```
system_instruction = """
당신은 10년 경력의 대기업 인사담당자이자 면접 코치입니다.
자소서와 채용공고를 분석해 실제 면접에서 나올 법한 질문을 생성하고,
지원자의 답변에 구체적이고 건설적인 피드백을 제공합니다.
```

다음 원칙을 따르세요:

- 예상 질문은 채용공고의 핵심 역량과 자소서 내용을 반영
- 답변 피드백은 STAR 기법(상황-과제-행동-결과) 기준으로 평가
- 개선이 필요한 부분은 구체적인 개선 예시와 함께 제시
- 지원자가 자신감을 가질 수 있도록 강점도 함께 언급

```
"""
```

LangChain + Streamlit 연결 구조

프로젝트 1 : 면접 준비 에이전트 구현

■ 에이전트 로직(1)

```
# agent.py — 에이전트 로직
```

```
from langchain.agents import create_tool_calling_agent, AgentExecutor
from langchain.memory import ConversationBufferMemory
from langchain.prompts import ChatPromptTemplate
from langchain_openai import ChatOpenAI
from dotenv import load_dotenv
```

```
load_dotenv()
```

```
llm = ChatOpenAI(model="gpt-4o-mini")
```

```
memory = ConversationBufferMemory(
    memory_key="chat_history",
    return_messages=True
)
...
```

수업용 자료 | 무단 배포 금지

LangChain + Streamlit 연결 구조

프로젝트 1 : 면접 준비 에이전트 구현

■ 에이전트 로직(2)

```
...
prompt = ChatPromptTemplate.from_messages([
    ("system", """
당신은 10년 경력의 대기업 인사담당자이자 면접 코치입니다.
자소서와 채용공고를 분석해 실제 면접에서 나올 법한 질문을 생성하고
답변에 구체적이고 건설적인 피드백을 제공합니다.
- 예상 질문은 채용공고의 핵심 역량과 자소서 내용을 반영
- 답변 피드백은 STAR 기법 기준으로 평가
- 개선이 필요한 부분은 구체적인 개선 예시와 함께 제시
- 지원자가 자신감을 가질 수 있도록 강점도 함께 언급
"""),
    ("placeholder", "{chat_history}"),
    ("human", "{input}"),
    ("placeholder", "{agent_scratchpad}")
])

agent = create_tool_calling_agent(llm, tools, prompt)
executor = AgentExecutor(agent=agent, tools=tools, memory=memory, verbose=True)

def run_interview_coach(user_input: str) -> str:
    """에이전트 실행 함수 — Streamlit에서 호출"""
    result = executor.invoke({"input": user_input})
    return result["output"]
```

LangChain + Streamlit 연결 구조

프로젝트 1 : 면접 준비 에이전트 구현

■ Streamlit UI(1)

```
# app.py
import streamlit as st
from agent import run_interview_coach

st.title("면접 준비 에이전트")
st.markdown("자소서와 채용공고를 입력하고 면접 준비를 시작하세요!")

with st.sidebar:
    st.header("정보 입력")
    resume = st.text_area("자소서", height=200,
                           placeholder="자소서 내용을 입력하세요")
    job_posting = st.text_area("채용광고", height=200,
                                placeholder="채용광고 내용을 입력하세요")
    if st.button("정보 저장"):
        st.session_state.resume = resume
        st.session_state.job_posting = job_posting
        st.success("저장되었습니다!")
...
```

수업용 자료 | 무단 배포 금지

LangChain + Streamlit 연결 구조

프로젝트 1 : 면접 준비 에이전트 구현

■ Streamlit UI(2)

```
...
if "messages" not in st.session_state:
    st.session_state.messages = []

for message in st.session_state.messages:
    with st.chat_message(message["role"]):
        st.write(message["content"])

if user_input := st.chat_input("질문을 입력하세요 (예: 면접 질문 생성해줘, 1번 답변 피드백해줘)":
    with st.chat_message("user"):
        st.write(user_input)
    st.session_state.messages.append({"role": "user", "content": user_input})

    with st.spinner("면접 코치가 분석 중입니다..."):
        response = run_interview_coach(user_input)

    with st.chat_message("assistant"):
        st.write(response)
    st.session_state.messages.append({"role": "assistant", "content": response})
```

수업용 자료 | 무단 배포 금지

실습: 에이전트 만들기

프로젝트 1 : 면접 준비 에이전트 구현

■ 실습의 목적:

- LangChain AgentExecutor와 Streamlit UI를 연결해 웹 서비스로 구현
- 에이전트 로직(agent.py)과 UI(app.py)를 분리해서 관리하는 방식 체험
- 브라우저에서 직접 사용할 수 있는 완성된 서비스 제작

🔍 확인해 볼 것

- streamlit run app.py 실행 후 브라우저에서 서비스가 정상 작동하는지 확인
- 자소서와 채용공고를 입력하고 "면접 질문 생성해줘" 요청 후 결과 확인
- 생성된 질문에 답변을 입력하고 피드백이 STAR 기법 기준으로 제공되는지 확인

실습: 에이전트 만들기

프로젝트 1 : 면접 준비 에이전트 구현

- 아래 프롬프트를 평소 사용하는 AI(ChatGPT, Claude 등)에 입력하고, 생성된 코드를 VSCode에서 실행해보세요.

💡 프롬프트 예시(1)

개발 환경은 Miniconda 가상환경(Python 3.11)이며 VSCode 터미널에서 실행합니다.
Python으로 LangChain과 Streamlit을 활용한 면접 준비 에이전트를 만들어줘. 조건은 다음과 같아.

- agent.py와 app.py 두 파일로 분리해서 구현
- agent.py 조건:
 - .env 파일에서 OpenAI API Key를 불러오도록 구현
 - @tool 데코레이터로 아래 세 가지 Tool 정의:
 - analyze_resume_and_job(resume, job_posting): 자소서와 채용공고 분석
 - generate_interview_questions(analysis): 예상 면접 질문 5가지 생성
 - feedback_answer(question, answer): STAR 기법 기준 답변 피드백
 - system_instruction은 아래 내용을 그대로 사용:
당신은 10년 경력의 대기업 인사담당자이자 면접 코치입니다.
자소서와 채용공고를 분석해 실제 면접에서 나올 법한 질문을 생성하고
답변에 구체적이고 건설적인 피드백을 제공합니다.
 - 예상 질문은 채용공고의 핵심 역량과 자소서 내용을 반영
 - 답변 피드백은 STAR 기법 기준으로 평가
 - 개선이 필요한 부분은 구체적인 개선 예시와 함께 제시
 - 지원자가 자신감을 가질 수 있도록 강점도 함께 언급

실습: 에이전트 만들기

프로젝트 1 : 면접 준비 에이전트 구현

- 아래 프롬프트를 평소 사용하는 AI(ChatGPT, Claude 등)에 입력하고, 생성된 코드를 VSCode에서 실행해보세요.

💡 프롬프트 예시(2)

- ConversationBufferMemory로 대화 기록 유지
- create_tool_calling_agent와 AgentExecutor로 에이전트 구성
- run_interview_coach(user_input) 함수로 외부에서 호출 가능하도록 구현
- gpt-4o-mini 모델 사용
- app.py 조건:
 - Streamlit으로 채팅 인터페이스 구현
 - 사이드바에 자소서와 채용공고 입력 영역 추가
 - "정보 저장" 버튼 클릭 시 session_state에 저장
 - st.session_state로 대화 기록 유지
 - st.spinner로 로딩 표시
 - agent.py의 run_interview_coach() 함수 호출해서 응답 생성

더 발전시켜보기

프로젝트 1 : 면접 준비 에이전트 구현

- **완성한 에이전트에 아래 기능 중 하나를 골라 직접 추가해보세요.**
앞에서 했던 것처럼 AI에게 추가 요청을 넣어서 코드를 발전시켜 나가면 됩니다.

추가 기능	프롬프트 힌트
대화 기록 초기화 버튼	"사이드바에 대화 기록을 초기화하는 버튼을 추가해줘"
면접 결과 저장	"생성된 질문과 피드백을 txt 파일로 다운로드할 수 있는 버튼을 추가해줘"
모의 면접 모드	"질문을 하나씩 순서대로 제시하고 답변을 받아 최종 종합 피드백을 제공하는 모의 면접 모드를 추가해줘"

프로젝트 2 : AI 학습 플래너 서비스 구현



오늘 만들 것

프로젝트 2 : AI 학습 플래너 서비스 구현

- 과목, 시험 일정, 하루 학습 가능 시간을 입력하면 맞춤형 학습 계획을 수립하고 관련 자료를 검색해주는 AI 학습 플래너 서비스를 Streamlit 웹 서비스로 구현.
- 면접 준비 에이전트와의 차이점:
 - LangChain AgentExecutor → LangGraph Supervisor 패턴으로 발전
 - 단일 역할 에이전트 → 여러 에이전트가 협업하는 구조
 - 같은 Streamlit UI 방식 유지 → 구조만 더 복잡해짐
- 완성 예시:

[사이드바]

과목: 자료구조

시험 날짜: 2026-08-15

하루 학습 가능 시간: 3시간

시험까지 37일 남았습니다.

[채팅 화면]

사용자: 학습 계획 짜줘

에이전트: 37일 동안 하루 3시간 기준 학습 계획입니다.

1주차: 배열, 연결 리스트 기초...

사용자: 관련 자료 찾아줘

에이전트: 자료구조 관련 최신 자료를 검색했습니다...

에이전트 설계

프로젝트 2 : AI 학습 플래너 서비스 구현

- **에이전트 이름:** AI 학습 플래너 서비스
- 에이전트 목표:** 과목, 시험 일정, 학습 가능 시간을 입력하면 맞춤형 학습 계획을 수립하고 관련 자료를 검색해서 제공
- 타겟 사용자:** 시험 준비 중인 대학생

구성요소	내용
LLM	gpt-4o-mini / "당신은 대학생 전문 학습 코치입니다. 학생의 과목, 시험 일정, 학습 가능 시간을 바탕으로 현실적이고 구체적인 학습 계획을 수립하고 동기부여가 되는 피드백을 제공합니다."
Tool	학습 계획 생성 함수, 뉴스·자료 검색 함수(NewsData API), 진도 체크 및 피드백 함수
Memory	과목, 시험 날짜, 학습 목표, 대화 기록 유지 (ConversationBufferMemory)
Workflow	사용자 요청 분석(Supervisor) → 계획 생성/자료 검색/진도 체크 → 결과 취합 → Streamlit UI 출력

- **구현 계획:**
 - 사용할 라이브러리: langchain, langchain-openai, langgraph, streamlit, python-dotenv, requests
 - 필요한 외부 API: OpenAI API, NewsData API
 - 구현 방식: LangGraph (Supervisor 패턴) + Streamlit (웹 UI)

LangGraph + Streamlit 연결 구조

프로젝트 2 : AI 학습 플래너 서비스 구현

■ 에이전트 로직 (LangGraph) (1)

```
# agent.py from langgraph.graph import StateGraph, END
from langchain_openai import ChatOpenAI
from typing import TypedDict
from dotenv import load_dotenv
import os
```

```
load_dotenv()
llm = ChatOpenAI(model="gpt-4o-mini")
```

```
class PlannerState(TypedDict):
    user_input: str      # 사용자 입력
    subject: str         # 과목명
    exam_date: str       # 시험 날짜
    study_hours: int     # 하루 학습 가능 시간
    next_agent: str      # 다음 실행할 에이전트
    plan_result: str     # 학습 계획 결과
    news_result: str     # 관련 자료 검색 결과
    progress_result: str # 진도 체크 결과
    final_answer: str    # 최종 답변
    ...
```

수업용 자료 | 무단 배포 금지

LangGraph + Streamlit 연결 구조

프로젝트 2 : AI 학습 플래너 서비스 구현

■ 에이전트 로직 (LangGraph) (2)

```
def supervisor_node(state: PlannerState) -> PlannerState:
```

```
    """LLM이 사용자 요청을 분석해서 적절한 에이전트 선택"""
```

```
    prompt = f"""
```

```
    사용자 요청: {state["user_input"]}
```

```
    아래 에이전트 중 가장 적합한 하나를 선택해서 이름만 답해줘.
```

```
    - plan: 학습 계획 수립이나 일정 관리가 필요한 경우
```

```
    - news: 학습 자료 검색이나 관련 정보가 필요한 경우
```

```
    - progress: 학습 진도 체크나 피드백이 필요한 경우
```

```
    답변은 반드시 plan, news, progress 중 하나만 출력.
```

```
    """
```

```
    response = llm.invoke(prompt)
```

```
    state["next_agent"] = response.content.strip()
```

```
    return state
```

```
def route_agent(state: PlannerState) -> str:
```

```
    return state["next_agent"]
```

```
def plan_agent_node(state: PlannerState) -> PlannerState:
```

```
    """학습 계획을 생성하는 에이전트 노드"""
```

```
    prompt = f"{state['subject']} 과목을 {state['exam_date']}까지 하루 {state['study_hours']}시간씩 공부할 계획을 세워줘"
```

```
    response = llm.invoke(prompt)
```

```
    state["plan_result"] = response.content
```

```
    return state
```

LangGraph + Streamlit 연결 구조

프로젝트 2 : AI 학습 플래너 서비스 구현

■ 에이전트 로직 (LangGraph) (3)

```
def news_agent_node(state: PlannerState) -> PlannerState:
    """NewsData API로 관련 자료를 검색하는 에이전트 노드"""
    # 실제 NewsData API 호출
    state["news_result"] = f"{state['subject']} 관련 자료 검색 결과 (임시)"
    return state

def progress_agent_node(state: PlannerState) -> PlannerState:
    """진도를 체크하고 피드백을 제공하는 에이전트 노드"""
    prompt = f"{state['subject']} 학습 진도 체크: {state['user_input']}"
    response = llm.invoke(prompt)
    state["progress_result"] = response.content
    return state

def final_node(state: PlannerState) -> PlannerState:
    """각 에이전트 결과를 취합해 최종 답변 생성"""
    results = []
    if state["plan_result"]:
        results.append(f"[학습 계획]\n{state['plan_result']}")
    if state["news_result"]:
        results.append(f"[관련 자료]\n{state['news_result']}")
    if state["progress_result"]:
        results.append(f"[진도 피드백]\n{state['progress_result']}")
    state["final_answer"] = "\n\n".join(results)
    return state
```

수업용 자료 | 무단 배포 금지

LangGraph + Streamlit 연결 구조

프로젝트 2 : AI 학습 플래너 서비스 구현

■ 에이전트 로직 (LangGraph) (4)

```
# 그래프 구성
graph = StateGraph(PlannerState)
graph.add_node("supervisor", supervisor_node)
graph.add_node("plan", plan_agent_node)
graph.add_node("news", news_agent_node)
graph.add_node("progress", progress_agent_node)
graph.add_node("final", final_node)

graph.set_entry_point("supervisor")
graph.add_conditional_edges("supervisor", route_agent, {
    "plan": "plan",
    "news": "news",
    "progress": "progress"
})
graph.add_edge("plan", "final")
graph.add_edge("news", "final")
graph.add_edge("progress", "final")
graph.add_edge("final", END)
```

수업용 자료 | 무단 배포 금지

LangGraph + Streamlit 연결 구조

프로젝트 2 : AI 학습 플래너 서비스 구현

■ 에이전트 로직 (LangGraph) (5)

```
app = graph.compile()

def run_planner(user_input: str, subject: str, exam_date: str, study_hours: int) -> str:
    """에이전트 실행 함수 — Streamlit에서 호출"""
    result = app.invoke({
        "user_input": user_input,
        "subject": subject,
        "exam_date": exam_date,
        "study_hours": study_hours,
        "next_agent": "",
        "plan_result": "",
        "news_result": "",
        "progress_result": "",
        "final_answer": ""
    })
    return result["final_answer"]
```

수업용 자료 | 무단 배포 금지

LangGraph + Streamlit 연결 구조

프로젝트 2 : AI 학습 플래너 서비스 구현

■ Streamlit UI (1)

```
# app.py
import streamlit as st
from agent import run_planner
import datetime

st.title("AI 학습 플래너 서비스")
st.markdown("과목과 시험 일정을 입력하면 맞춤형 학습 계획을 제공합니다.")

with st.sidebar:
    st.header("학습 정보 입력")
    subject = st.text_input("과목명", placeholder="예: 자료구조")
    exam_date = st.date_input("시험 날짜", datetime.date.today())
    study_hours = st.slider("하루 학습 가능 시간(시간)", 1, 12, 3)
    days_left = (exam_date - datetime.date.today()).days
    st.markdown(f"시험까지 **{days_left}일** 남았습니다.")

if "messages" not in st.session_state:
    st.session_state.messages = []

for message in st.session_state.messages:
    with st.chat_message(message["role"]):
        st.write(message["content"])
```

LangGraph + Streamlit 연결 구조

프로젝트 2 : AI 학습 플래너 서비스 구현

■ Streamlit UI (2)

```
if user_input := st.chat_input("질문을 입력하세요 (예: 학습 계획 짜줘, 관련 자료 찾아줘, 오늘 진도 체크해줘)":
    with st.chat_message("user"):
        st.write(user_input)
    st.session_state.messages.append({"role": "user", "content": user_input})

    with st.spinner("AI 플래너가 분석 중입니다..."):
        response = run_planner(
            user_input=user_input,
            subject=subject,
            exam_date=str(exam_date),
            study_hours=study_hours
        )

    with st.chat_message("assistant"):
        st.write(response)
    st.session_state.messages.append({"role": "assistant", "content": response})
```

수업용 자료 | 무단 배포 금지

실습: AI 학습 플래너 서비스 만들기

프로젝트 2 : AI 학습 플래너 서비스 구현

■ 실습의 목적:

- LangGraph 에이전트와 Streamlit UI를 연결해 완성도 높은 웹 서비스 구현
- LangChain 방식과 LangGraph 방식의 구조적 차이 체감
- 완성된 서비스 제작

🔍 확인해 볼 것

- streamlit run app.py 실행 후 브라우저에서 서비스가 정상 작동하는지 확인
- 과목과 시험 날짜를 바꿔가며 학습 계획이 어떻게 달라지는지 확인
- "학습 계획 짜줘", "관련 자료 찾아줘", "오늘 진도 체크해줘" 등 다양한 요청으로 Supervisor가 올바른 에이전트를 선택하는지 확인

실습: AI 학습 플래너 서비스 만들기

프로젝트 2 : AI 학습 플래너 서비스 구현

- 아래 프롬프트를 평소 사용하는 AI(ChatGPT, Claude 등)에 입력하고, 생성된 코드를 VSCode에서 실행해보세요.

💡 프롬프트 예시(1)

개발 환경은 Miniconda 가상환경(Python 3.11)이며 VSCode 터미널에서 실행합니다.
Python으로 LangGraph와 Streamlit을 활용한 AI 학습 플래너 서비스를 만들어줘. 조건은 다음과 같아.

- agent.py와 app.py 두 파일로 분리해서 구현
- agent.py 조건:
 - .env 파일에서 OpenAI API Key와 NEWSDATA_API_KEY를 불러오도록 구현
 - TypedDict로 PlannerState 정의 (user_input, subject, exam_date, study_hours, next_agent, plan_result, news_result, progress_result, final_answer 필드 포함)
 - Supervisor 패턴으로 아래 세 가지 에이전트 노드 구현:
 - plan_agent_node: 과목, 시험 날짜, 하루 학습 가능 시간을 바탕으로 일별 학습 계획 생성
 - news_agent_node: NewsData API로 과목 관련 최신 학습 자료 검색
 - progress_agent_node: 오늘 학습한 내용을 입력받아 진도 체크 및 피드백 제공
 - 각 에이전트 완료 후 final_node로 이동해 결과 취합
 - run_planner(user_input, subject, exam_date, study_hours) 함수로 외부에서 호출 가능하도록 구현
 - gpt-4o-mini 모델 사용

실습: AI 학습 플래너 서비스 만들기

프로젝트 2 : AI 학습 플래너 서비스 구현

- 아래 프롬프트를 평소 사용하는 AI(ChatGPT, Claude 등)에 입력하고, 생성된 코드를 VSCode에서 실행해보세요.

💡 프롬프트 예시(2)

- app.py 조건:
 - Streamlit으로 채팅 인터페이스 구현
 - 사이드바에 과목명, 시험 날짜(date_input), 하루 학습 가능 시간(slider) 입력 영역 추가
 - 시험까지 남은 날짜를 사이드바에 자동으로 계산해서 표시
 - st.session_state로 대화 기록 유지
 - st.spinner로 로딩 표시
 - agent.py의 run_planner() 함수 호출해서 응답 생성

더 발전시켜보기

프로젝트 2 : AI 학습 플래너 서비스 구현

- **완성한 에이전트에 아래 기능 중 하나를 골라 직접 추가해보세요.**
앞에서 했던 것처럼 AI에게 추가 요청을 넣어서 코드를 발전시켜 나가면 됩니다.

추가 기능	프롬프트 힌트
대화 기록 초기화 버튼	"사이드바에 대화 기록을 초기화하는 버튼을 추가해줘"
학습 계획 파일 저장	"생성된 학습 계획을 txt 파일로 다운로드할 수 있는 버튼을 추가해줘"
여러 과목 동시 관리	"여러 과목을 동시에 입력하고 과목별 학습 시간을 배분하는 기능을 추가해줘"

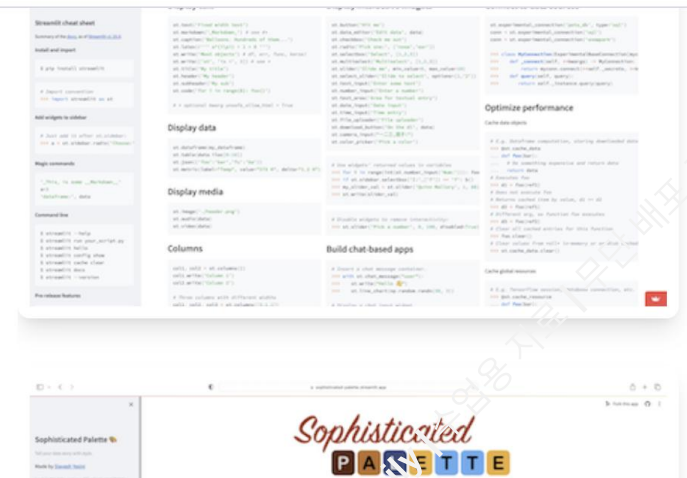
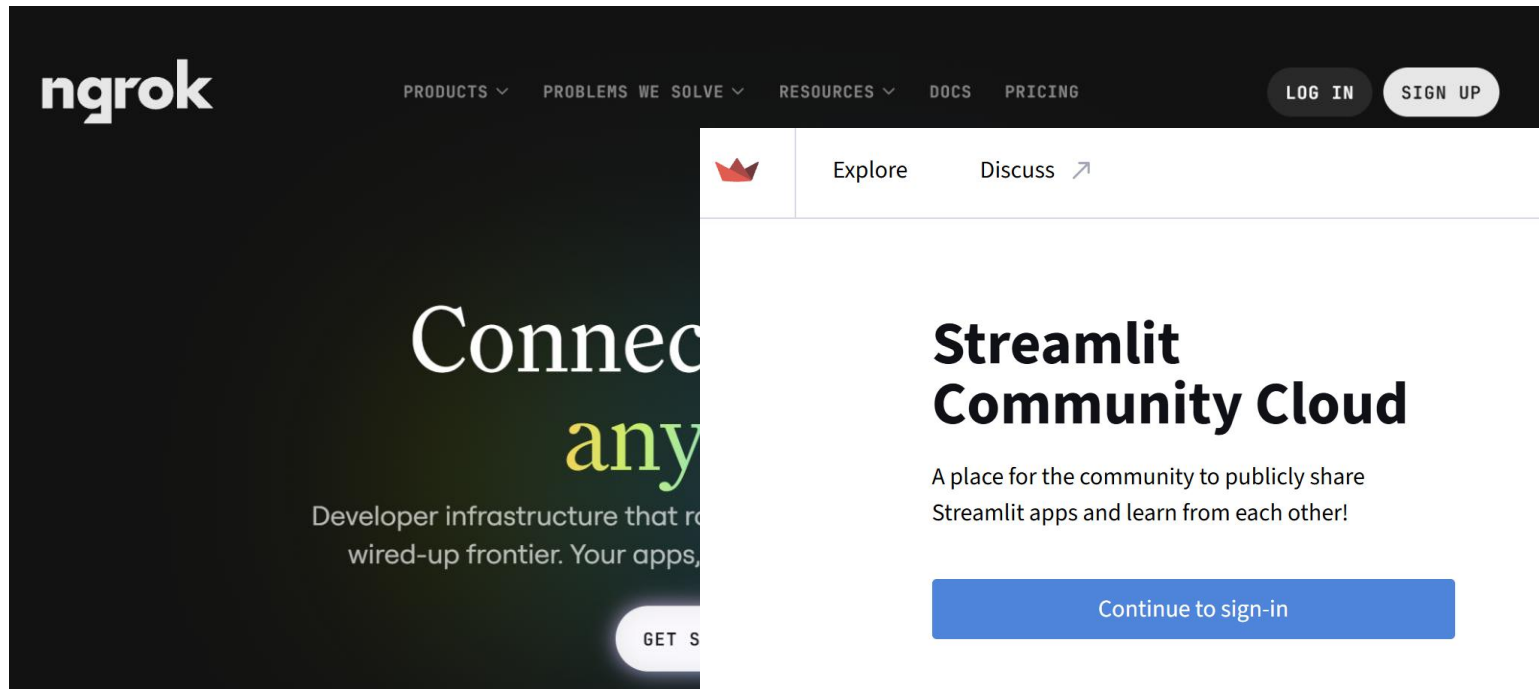
배포



© Emily / > 2020 / 11 / 11

- Streamlit 앱을 외부에 공개하는 방법. 시간과 상황에 맞게 선택.

방법	난이도	비용	GitHub 필요	특징
Streamlit Cloud	☆☆	무료	✓ 필요	가장 안정적
ngrok	☆	무료	✗ 불필요	로컬 실행 중에만 접속 가능, 빠른 공유에 적합



- 방법 1 — ngrok (빠른 공유용)
- 사전 준비 (최초 1회):
 - ngrok.com 접속 및 회원가입
 - 대시보드에서 authtoken 복사
 - 아래 명령어로 토큰 등록:

```
python -c "from pyngrok import ngrok; ngrok.set_auth_token('your-authtoken')"
```

- 실행 방법

```
# ngrok_run.py — 프로젝트 폴더에 미리 저장
from pyngrok import ngrok

tunnel = ngrok.connect(8501)
print(f"공개 URL: {tunnel.public_url}")

input("종료하려면 Enter를 누르세요...")
ngrok.kill()
```

```
# 터미널 1에서 Streamlit 실행
streamlit run app.py
```

```
# VSCode에서 새 터미널 열기 (+ 버튼)
# 터미널 2에서 실행
python ngrok_run.py
```

- ngrok 실행 후 출력된 URL을 공유하면 다른 사람도 바로 접속 가능.

■ 방법 2 — Streamlit Cloud (영구 배포)

- 1) share.streamlit.io 접속 및 GitHub 계정으로 로그인
- 2) "Create app" 클릭
- 3) 배포할 GitHub 저장소, 브랜치, 메인 파일(app.py) 선택
- 4) "Advanced settings" → Secrets에 .env 내용 입력

```
OPENAI_API_KEY=sk-xxxxxxxxxxxxxxxxxxxxxxxxxxxxx  
NEWSDATA_API_KEY=pub_xxxxxxxxxxxxxxxxxxxxxxxxxxxxx
```

- 5) "Deploy" 클릭 → 공유 가능한 URL 생성

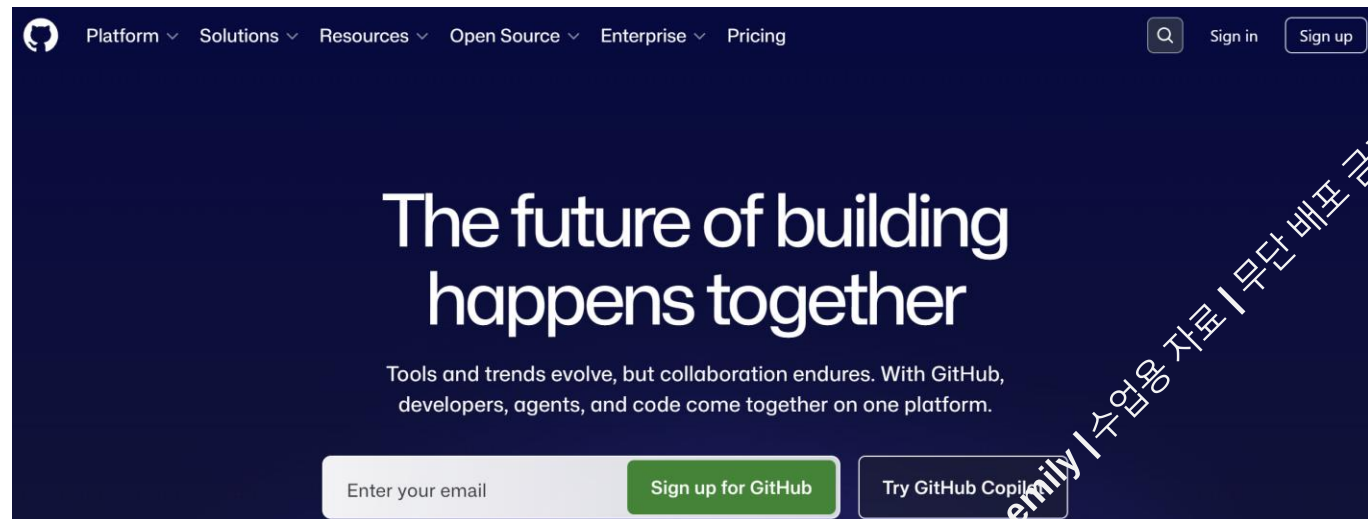
➔ API Key는 반드시 Streamlit Cloud의 Secrets 기능으로 관리.

GitHub이란?

배포

- GitHub는 코드를 온라인에 저장하고 관리하는 플랫폼.
개발자들이 프로젝트를 공유하고 협업하는 공간으로, 취업 시 포트폴리오로 활용 가능.
- GitHub이 중요한 이유:
 - 포트폴리오: 채용 담당자가 지원자의 GitHub을 통해 실력을 확인하는 경우가 많음
 - 버전 관리: 코드 변경 이력을 추적하고 이전 버전으로 되돌릴 수 있음
 - 협업: 팀 프로젝트에서 여러 사람이 동시에 코드 작업 가능
 - 배포 연동: Streamlit Cloud 등 배포 플랫폼과 연동 가능

➔ Streamlit Cloud가 GitHub 저장소에서
코드를 가져오는 방식이라 필요



GitHub 기본 설정 및 코드 올리기

배포

- **Streamlit Cloud 배포를 위해 코드를 GitHub에 올려야 함. 아래 순서대로 진행.**
- **사전 준비 (최초 1회)**

- 1) github.com 접속 및 회원가입
- 2) git.scm.com에서 Git 설치 (Windows)
- 3) VSCode 터미널에서 Git 사용자 정보 등록:

```
git config --global user.name "이름"  
git config --global user.email "이메일"
```

- 4) github.com에서 새 저장소(Repository) 생성: "New repository" 클릭
 - 저장소 이름 입력 (예: ai-learning-planner)
 - Public 선택
 - "Create repository" 클릭

GitHub 기본 설정 및 코드 올리기

배포

- 코드 올리기(프로젝트 폴더에서 실행)

```
# 1. Git 초기화
```

```
git init
```

```
# 2. .gitignore 파일 생성 — API Key 등 민감 정보 제외
```

```
# .gitignore 파일에 아래 내용 추가:
```

```
# .env
```

```
# __pycache__/
```

```
# *.pyc
```

```
# 3. 파일 추가
```

```
git add .
```

```
# 4. 커밋
```

```
git commit -m "AI 학습 플래너 서비스 완성"
```

```
# 5. GitHub 저장소 연결 및 푸시
```

```
git remote add origin https://github.com/사용자명/저장소명.git
```

```
git push -u origin main
```

- .env 파일은 반드시 .gitignore에 추가해서 API Key가 GitHub에 올라가지 않도록 주의.

requirements.txt 작성

배포

- requirements.txt는 이 프로젝트를 실행하는 데 필요한 라이브러리 목록을 기록한 파일.
다른 사람이 내 코드를 받아서 실행할 때, 또는 Streamlit Cloud에 배포할 때 어떤 라이브러리를 설치해야 하는지 자동으로 알려주는 역할을 함.(Streamlit Cloud로 영구 배포할 때만 작성.)
- requirements.txt가 없으면:
 - Streamlit Cloud가 어떤 라이브러리를 설치해야 할지 몰라서 배포 실패
 - 다른 사람이 내 코드를 받아도 실행 환경을 직접 구성해야 해서 불편

```
# 현재 가상환경에 설치된 라이브러리 목록 자동 생성  
pip freeze > requirements.txt
```

```
# requirements.txt 예시  
langchain  
langchain-openai  
langgraph  
streamlit  
python-dotenv  
requests  
pyngrok
```

- pip freeze를 하면 가상환경에 설치된 모든 라이브러리가 포함되어 목록이 길어질 수 있음. 실제로 사용한 라이브러리만 남기고 정리하는 것이 좋음.

5일간의 여정 정리

배포

- 5일 동안 Python 기초에서 시작해 멀티 에이전트 서비스 배포까지 완성.
1차시에서 배운 LLM·Tool·Memory·Workflow 4가지 프레임워크가 모든 에이전트의 기반이 됨.

Day	차시	핵심 내용	만든 것
Day 1	1~6차시	Python 기초	챗봇 루프, 에이전트 클래스
Day 2	7~12차시	LLM API 연동	MBTI 추천, 뉴스 검색·요약 에이전트
Day 3	13~18차시	LangChain	여행 플래너, 자소서 리뷰 에이전트
Day 4	19~24차시	LangGraph	RAG 에이전트, 멀티 에이전트
Day 5	25~30차시	최종 프로젝트	면접 준비 에이전트, AI 학습 플래너 서비스

- 이번 과정은 AI 에이전트 개발의 시작점.
개발자가 아니더라도 AI를 직접 만들고 활용하는 역량은 어떤 직무에서든 강력한 경쟁력이 됨.

직무/분야	AI 에이전트 활용 방향
마케팅·콘텐츠	트렌드 뉴스 수집·요약 에이전트, SNS 콘텐츠 자동 생성
기획·전략	시장 조사 에이전트, 보고서 초안 작성 자동화
인사·교육	채용 공고 분석 에이전트, 학습 플래너 서비스
연구·분석	논문 요약 에이전트, 데이터 분석 자동화
개발	LangChain, LangGraph 심화, 클라우드 배포

- AI 활용 능력이 높아질수록 구현의 장벽이 낮아짐.
이제 중요한 것은 어떤 문제를 해결할 것인가를 정의하는 능력.
오늘 배운 설계 사고와 에이전트 구조를 바탕으로 나만의 문제를 해결하는 도구를 계속 만들어 나가세요.

오늘 하루도 수고하셨습니다